

DeforestWatch-DRC

Synthèse technique illustrée : de l'image satellite au modèle prédictif

Ce document explique le fonctionnement interne du projet en trois temps : la collecte des images satellites, l'apprentissage automatique appliqué à ces images, et le stockage de l'ensemble des données produites.

Zone d'étude : Inongo, province du Mai-Ndombe, République Démocratique du Congo. Forêt tropicale humide équatoriale du Bassin du Congo, sur un carré d'environ 50 kilomètres de côté, observé de 2015 à 2025.

Chaque figure de ce document est produite par le code du projet lui-même (scripts/figures_synthese.py), à partir des mêmes fonctions que celles utilisées par le pipeline.

Sommaire

1. Le projet en bref
2. La chaîne de traitement, vue d'ensemble
3. La collecte des images satellites
4. Du pixel brut au vecteur de caractéristiques
5. Le Machine Learning
6. Le stockage des données
7. La bascule démonstration / données réelles
8. Questions de soutenance
9. Aide-mémoire des commandes

1. Le projet en bref

La République Démocratique du Congo abrite la deuxième plus grande forêt tropicale humide du monde. Elle recule, et le suivi de terrain seul ne permet pas de mesurer ce recul à l'échelle d'une province. Les satellites d'observation de la Terre passent au-dessus de la même zone tous les cinq jours et fournissent gratuitement leurs images.

DeforestWatch-DRC exploite ces images pour répondre à quatre questions :

1. Où se trouve la forêt aujourd'hui ? C'est un problème de classification d'images.
2. Combien en a-t-on perdu, année par année ? C'est une comparaison de classifications successives.
3. Où va-t-elle probablement disparaître dans les deux ans ? C'est un problème de prédiction.
4. Comment rendre tout cela consultable ? C'est le rôle de l'API, du dashboard et du frontend.

Les cinq classes de couverture du sol

Toute la chaîne repose sur ces cinq classes. Un pixel appartient toujours à une et une seule d'entre elles.

Code	Classe	Ce que cela représente sur le terrain
0	Forêt dense	Canopée fermée, forêt primaire ou secondaire mature
1	Forêt dégradée	Couvert ouvert, exploitation sélective, lisière
2	Agriculture / sol nu	Champ, jachère, brûlis, sol mis à nu
3	Eau	Rivière, lac, zone inondée
4	Zone urbaine / bâti	Village, ville, infrastructure

La distinction entre forêt dense et forêt dégradée compte : la dégradation précède presque toujours la conversion complète en terre agricole. Détecter la dégradation, c'est détecter la déforestation avec un temps d'avance.

Le projet fonctionne dans deux modes. En mode démonstration, des données synthétiques réalistes permettent de tout exécuter sans compte Google Earth Engine. En mode réel, les mêmes traitements s'appliquent à de vraies images. La section 7 explique ce mécanisme.

2. La chaîne de traitement, vue d'ensemble

Avant d'entrer dans le détail, voici le trajet complet d'une donnée, depuis le capteur en orbite jusqu'à l'écran de l'utilisateur. Les sections suivantes reprennent chacune de ces étapes.

La chaîne complète, de l'orbite au dashboard



Figure 1. Les huit étapes de la chaîne de traitement.

Deux points méritent d'être notés dès maintenant.

D'abord, les étapes 1 et 2 sont séparées. Google Earth Engine calcule sur ses propres serveurs et ne renvoie pas les pixels automatiquement : il faut lancer un export explicite. C'est ce qui évite de télécharger des téraoctets d'archives.

Ensuite, à partir de l'étape 3, tout le code ignore d'où viennent les images. Qu'elles soient réelles ou simulées, elles arrivent sous la même forme : un tableau numpy de dimensions hauteur x largeur x bandes.

3. La collecte des images satellites

3.1 Pourquoi Google Earth Engine

Une seule scène Sentinel-2 pèse environ 700 Mo. Sur onze années et une zone de 50 kilomètres, avec un passage tous les cinq jours, l'archive complète représente plusieurs téraoctets. La télécharger serait absurde.

Google Earth Engine (GEE) héberge cette archive et permet d'y appliquer des traitements côté serveur. On décrit le calcul voulu, Google l'exécute sur son infrastructure, et seul le résultat final est exporté. Le code de collecte se trouve dans `src/data/gee_collector.py`.

3.2 Sentinel-2 : la source optique principale

Sentinel-2 est une paire de satellites européens du programme Copernicus. Ils photographient la Terre dans treize bandes spectrales, à une résolution de 10 mètres pour les principales. Un pixel de l'image correspond donc à un carré de 10 mètres de côté au sol.

Les six bandes retenues

Bande	Longueur d'onde	Pourquoi elle est utilisée
B2	490 nm, bleu	Correction atmosphérique, entre dans le calcul de l'EVI
B3	560 nm, vert	Détection des surfaces en eau
B4	665 nm, rouge	Fortement absorbé par la chlorophylle
B8	842 nm, proche infrarouge	Fortement réfléchi par une végétation saine
B11	1610 nm, SWIR 1	Sensible à l'humidité de la végétation
B12	2190 nm, SWIR 2	Sensible aux zones brûlées et au sol nu

Ces six bandes ne sont pas choisies au hasard. Chaque type de surface renvoie la lumière différemment selon la longueur d'onde, et cette empreinte est ce que le modèle apprend à reconnaître.

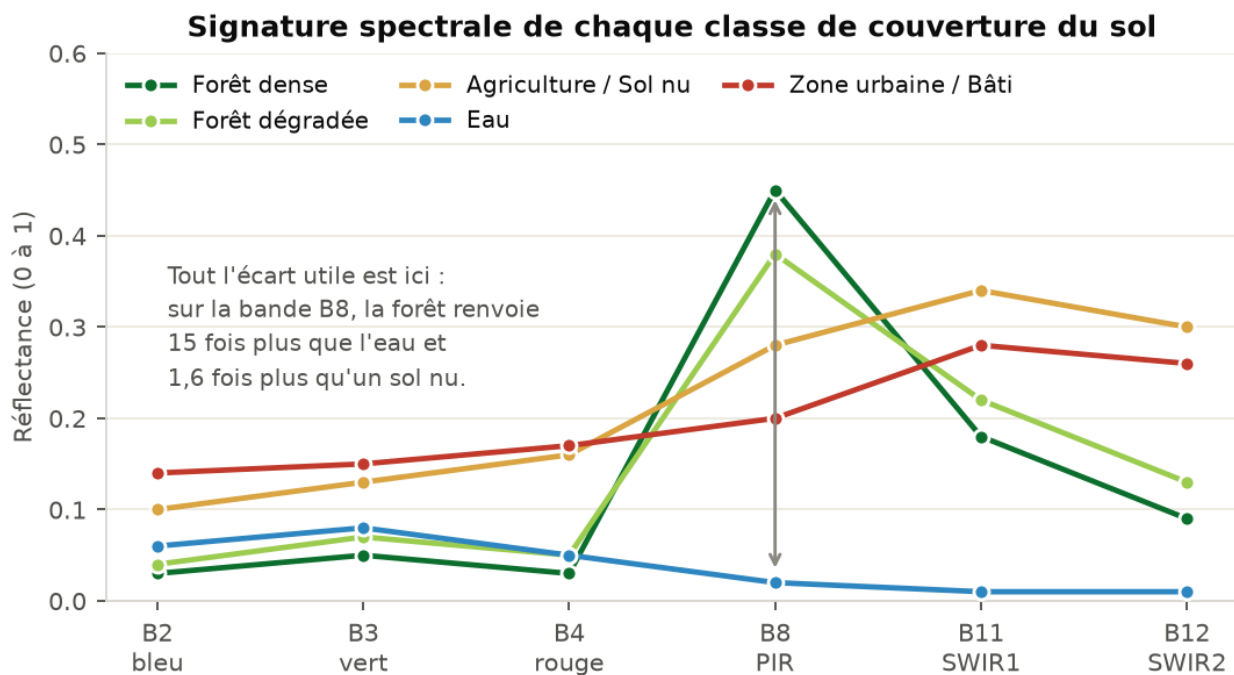


Figure 2. Signature spectrale moyenne de chaque classe. C'est la matière première de la classification.

La lecture de cette figure explique tout le principe de la télédétection. Une forêt dense absorbe presque tout le rouge, parce que sa chlorophylle s'en nourrit, et renvoie massivement le proche infrarouge, parce que la structure interne des feuilles le diffuse. Un sol nu ne fait ni l'un ni l'autre : sa courbe monte régulièrement. L'eau absorbe tout à partir du proche infrarouge et s'effondre à zéro.

Les courbes se croisent par endroits, notamment entre forêt dégradée et agriculture sur les bandes SWIR. C'est précisément là que le modèle a du travail, et c'est pourquoi une seule bande ne suffit pas.

3.3 Le masquage des nuages et le composite médian

Sous les tropiques, la couverture nuageuse est le principal obstacle. Une scène individuelle est souvent inexploitable à moitié. Deux mécanismes se combinent pour résoudre le problème.

Premier mécanisme : le masque SCL

Sentinel-2 fournit une bande supplémentaire appelée SCL (Scene Classification Layer), qui étiquette chaque pixel : végétation, sol nu, eau, nuage, ombre de nuage, neige. Le code écarte les pixels portant les codes 3 (ombre), 8, 9 et 10 (nuages de densité croissante) et 11 (neige). Ces pixels deviennent des trous dans l'image.

Deuxième mécanisme : la médiane temporelle

On ne travaille pas sur une scène, mais sur toutes les scènes de la saison sèche, du 1er juin au 30 septembre. Pour chaque pixel, on prend la valeur médiane de toutes les dates disponibles. Un nuage résiduel présent sur une seule date produit une valeur aberrante que la médiane écarte automatiquement.

Pourquoi la médiane : chaque nuage est écarté par les autres dates

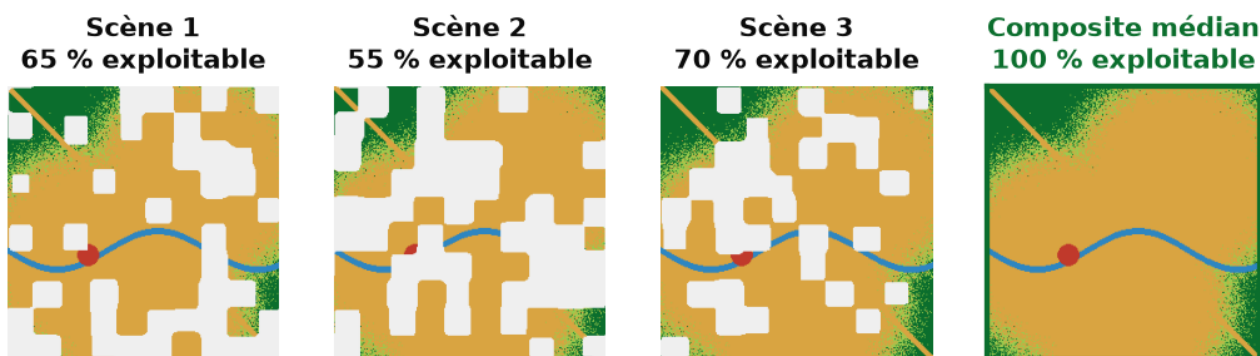


Figure 3. Trois scènes partiellement nuageuses, et le composite médian qui en résulte.

Le choix de la saison sèche n'est pas seulement pratique. Il garantit aussi que les images comparées d'une année sur l'autre le sont dans des conditions équivalentes : même angle solaire approximatif, même état phénologique de la végétation. Comparer une image de saison sèche à une image de saison des pluies produirait de faux changements.

La requête effectivement exécutée

```
ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(zone_etude) # emprise géographique
  .filterDate(f'{annee}-06-01', f'{annee}-09-30') # saison sèche
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 40))
  .map(masque_nuages_scl) # trous sur les nuages
  .median() # composite
  .select(['B2', 'B3', 'B4', 'B8', 'B11', 'B12'])
```

3.4 Sentinel-1 : le radar qui traverse les nuages

Même avec la médiane, certaines années restent trop nuageuses. Sentinel-1 apporte une solution complémentaire : c'est un radar, il émet sa propre onde et mesure ce qui revient. Les nuages sont transparents pour lui, et il fonctionne de nuit.

Le radar ne mesure pas la couleur mais la structure. Une forêt dense renvoie beaucoup d'énergie en polarisation croisée (VH), parce que l'onde rebondit dans le volume des branches. Un sol nu en renvoie peu. Une surface d'eau agit comme un miroir et renvoie presque rien vers le capteur.

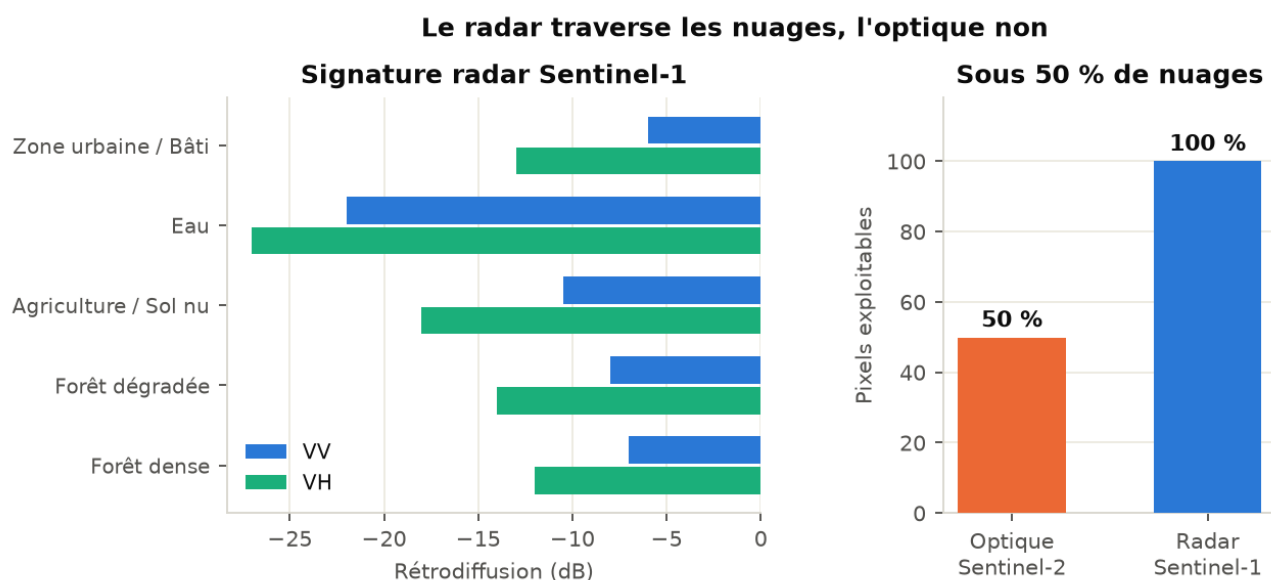


Figure 4. À gauche, la signature radar de chaque classe. À droite, le gain de couverture sous un ciel à moitié nuageux.

Le module `src/data/radar.py` calcule également le RVI (Radar Vegetation Index), défini comme 4 fois VH divisé par la somme VV plus VH, en puissance linéaire. Il donne un indicateur de densité de végétation indépendant de toute condition d'éclairage.

3.5 SRTM : le relief

La mission SRTM de la NASA a cartographié l'altitude de la quasi-totalité des terres émergées. On en dérive trois couches : l'altitude, la pente et l'exposition du versant.

Ces variables comptent parce que la déforestation suit la géographie. Un terrain plat et accessible est défriché en premier ; une pente forte est difficile à cultiver et protège la forêt plus longtemps. La topographie est donc un prédicteur, pas seulement une information de contexte.

3.6 Les données météorologiques

`src/data/weather_collector.py` collecte les précipitations et les températures mensuelles via OpenWeatherMap, avec un repli sur une climatologie équatoriale réaliste à deux saisons des pluies. Ces données servent à l'analyse contextuelle : une saison sèche prolongée augmente le risque de feux et facilite le défrichement.

3.7 Du serveur de Google au disque local

Le point souvent mal compris : appeler le code de collecte ne télécharge rien. GEE construit une description du calcul, et il faut lancer un export explicite pour obtenir les fichiers. C'est le rôle de `scripts/gee_export.py`, qui propose trois modes.

Mode	Commande	Usage
Export Drive	<code>gee_export --drive</code>	Production. Pleine résolution 10 m, vers Google Drive, à télécharger ensuite.
Téléchargement direct	<code>gee_export --download --scale 100</code>	Aperçu rapide. Résolution réduite, écriture directe dans <code>data/raw/</code> .
GeoTIFF de test	<code>gee_export --demo-geotiff</code>	Valider toute la chaîne réelle sans compte GEE. Écrit des GeoTIFF géoréférencés synthétiques.

Le troisième mode mérite une mention particulière. Il permet de vérifier que la lecture des GeoTIFF, l'alignement des grilles et l'entraînement sur données réelles fonctionnent, avant même de disposer des vraies images. C'est un filet de sécurité utile en démonstration.

Une limite à connaître et à assumer : dans l'état actuel du code, `get_annual_composite()` renvoie toujours un composite synthétique, même quand GEE est connecté. La seule voie vers de vraies données passe par l'export en GeoTIFF puis la lecture par `RasterSource`. C'est une décision d'architecture cohérente, pas un oubli : elle sépare nettement le calcul distant du traitement local.

4. Du pixel brut au vecteur de caractéristiques

Un modèle de Machine Learning ne comprend pas une image : il attend des nombres. L'étape de préparation des caractéristiques, ou features, consiste à transformer chaque pixel en une liste de valeurs qui décrivent ce qu'il est.

4.1 Les indices spectraux

Les six bandes brutes suffisent rarement. On les combine en indices, des rapports qui isolent une propriété physique précise et qui ont l'avantage d'être normalisés, donc peu sensibles aux variations d'éclairage.

Indice	Formule	Ce qu'il révèle
NDVI	$(B8 - B4) / (B8 + B4)$	Vigueur de la végétation. Proche de 1 sur forêt dense, négatif sur l'eau.
EVI	$2.5 (B8 - B4) / (B8 + 6 B4 - 7.5 B2 + 1)$	Même idée, mais sature moins sur les couverts très denses.
NDWI	$(B3 - B8) / (B3 + B8)$	Présence d'eau libre. Sépare rivières et zones inondées.
NBR	$(B8 - B12) / (B8 + B12)$	Zones brûlées. Le brûlis est le mode de défrichement dominant dans la zone.

Le NDVI est le plus connu, mais il sature : au-delà d'une certaine densité de canopée, il ne distingue plus rien. C'est exactement le cas d'une forêt équatoriale. D'où l'EVI, qui corrige ce défaut, et le NBR, qui repère les surfaces récemment brûlées que le NDVI seul confondrait avec du sol nu ordinaire.

Les indices spectraux font ressortir le front de déforestation

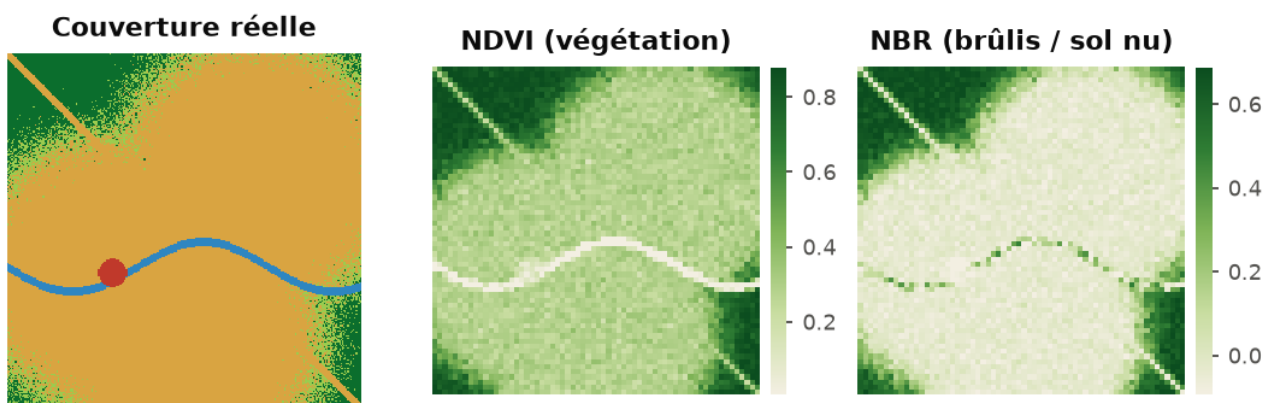


Figure 5. À gauche la couverture réelle, au centre et à droite les indices calculés sur le même composite.

Sur ces cartes, le front de déforestation apparaît sans qu'aucun modèle n'ait encore été entraîné. Les indices sont un calcul déterministe, pas un apprentissage. Ils fournissent au modèle une information déjà à moitié digérée, ce qui lui simplifie considérablement la tâche.

4.2 Le vecteur de treize caractéristiques

Un pixel = un vecteur de 13 nombres



Ordre figé dans `config/settings.py` (`FEATURE_NAMES`) et identique partout dans la chaîne

Figure 6. La composition du vecteur de features associé à chaque pixel.

L'ordre de ces treize valeurs est figé une fois pour toutes dans `config/settings.py`, sous le nom `FEATURE_NAMES`. Il est respecté partout : à la construction du dataset, à l'entraînement, à la prédiction, et à l'affichage des importances de variables.

Ce point paraît anodin et ne l'est pas. Si l'ordre changeait entre l'entraînement et la prédiction, le modèle interpréterait une altitude comme un NDVI. Il ne planterait pas : il donnerait simplement des résultats faux, silencieusement. Fixer l'ordre dans la configuration est une protection contre cette classe d'erreur.

L'ordre de grandeur

Sur une grille de 256 par 256 pixels, cela représente 65 536 pixels par année, donc autant de vecteurs de 13 nombres. Sur onze années, environ 720 000 exemples. Une vraie image à 10 mètres sur la même zone de 50 kilomètres en produirait 25 millions par année.

4.3 Le découpage en tuiles pour le réseau de neurones

Random Forest et XGBoost consomment des vecteurs de pixels indépendants. Le U-Net, lui, a besoin de voir une portion d'image. Le code (`src/preprocessing/feature_extraction.py`) découpe donc l'image en tuiles de 128 par 128 pixels, avec un recouvrement de 32 pixels entre tuiles voisines.

Le recouvrement a une justification précise. Un réseau convolutif est moins fiable sur les bords de son champ de vision, faute de contexte. Sans recouvrement, on verrait apparaître des coutures visibles aux jonctions de tuiles. Avec recouvrement, chaque zone frontalière est prédite plusieurs fois et les prédictions sont moyennées à la reconstruction.

4.4 Le nettoyage distribué avec PySpark

`src/preprocessing/spark_pipeline.py` applique un traitement distribué : suppression des valeurs manquantes, filtrage des valeurs aberrantes, statistiques par classe, normalisation par `StandardScaler`, puis export en Parquet partitionné par année. Si PySpark n'est pas installé, un repli pandas équivalent prend le relais.

Sur la taille actuelle du jeu de données, pandas suffirait. L'intérêt de Spark est de montrer que la chaîne tient à l'échelle : la même logique s'applique sans réécriture si la zone d'étude passe d'une province à un pays entier.

5. Le Machine Learning

5.1 Le principe de l'apprentissage supervisé

Tous les modèles du projet relèvent de l'apprentissage supervisé. Le principe tient en trois temps.

1. On dispose d'exemples étiquetés : des pixels dont on connaît déjà la classe, fournis par la vérité terrain.
2. Le modèle ajuste ses paramètres internes pour réduire l'écart entre ses prédictions et les étiquettes connues. Cet écart est mesuré par une fonction de perte.
3. On évalue le modèle sur des exemples qu'il n'a jamais vus. C'est la seule mesure honnête de sa capacité à généraliser.

Le troisième point est le plus important et le plus facile à saboter. Un modèle qui mémorise ses exemples d'entraînement obtient un score parfait dessus et s'effondre sur des données nouvelles. La section 5.7 explique la précaution particulière que demandent les données spatiales.

D'où viennent les étiquettes

En mode réel, la vérité terrain provient de deux sources : le produit Hansen Global Forest Change, qui fournit une carte mondiale de perte forestière annuelle depuis 2000, et une annotation manuelle de zones d'échantillon. Ces étiquettes sont déposées dans `data/raw/landcover/`.

5.2 Deux problèmes distincts

Il faut bien séparer les deux questions que le projet traite, car elles n'ont ni les mêmes entrées, ni les mêmes étiquettes, ni la même utilité.

	Problème A : classification	Problème B : prédiction du risque
Question	Qu'y a-t-il sur ce pixel aujourd'hui ?	Ce pixel de forêt sera-t-il défriché d'ici deux ans ?
Étiquette	Classe observée, 0 à 4	Comparaison entre l'année N et l'année N+1
Sortie	Carte de couverture du sol	Carte de risque graduée de 0 à 100
Modèles	Random Forest, XGBoost, U-Net	XGBoost binaire (ou GradientBoosting en repli)

5.3 Random Forest, la référence de départ

Un arbre de décision pose une suite de questions simples : le NDVI dépasse-t-il 0,7 ? La bande B11 est-elle inférieure à 0,2 ? Chaque réponse mène à une nouvelle question, jusqu'à une feuille qui donne la classe.

Un arbre seul est instable : changez quelques exemples d'entraînement et sa structure change complètement. La forêt aléatoire corrige ce défaut en entraînant 200 arbres, chacun sur un tirage différent des exemples et des variables, puis en faisant voter l'ensemble. Les erreurs individuelles se compensent.

Configuration retenue (config/settings.py) : 200 arbres, profondeur maximale 15, minimum 5 exemples pour scinder un noeud. La profondeur limitée est une protection contre le surapprentissage.

Son autre atout est l'interprétabilité. Le modèle indique quelles variables ont le plus contribué à ses décisions, ce qui permet de vérifier que le résultat est physiquement crédible : si l'aspect du versant dominait le NDVI dans la détection de forêt, il y aurait un problème.

5.4 XGBoost, l'affinage séquentiel

XGBoost construit lui aussi des arbres, mais en séquence et non en parallèle. Chaque nouvel arbre se concentre sur les exemples que les précédents ont mal classés. On corrige progressivement les erreurs résiduelles plutôt que de moyenner des avis indépendants.

Cette approche est généralement plus performante sur données tabulaires, au prix d'une sensibilité plus grande au réglage des hyperparamètres. Le taux d'apprentissage est fixé à 0,1 : chaque arbre ne corrige qu'une fraction de l'erreur, ce qui évite de sur-corriger sur du bruit.

Les deux modèles exposent volontairement la même interface (fit, predict, predict_proba, feature_importance), ce qui rend la comparaison directe et sans biais d'implémentation.

5.5 U-Net, le réseau qui voit le contexte

Random Forest et XGBoost jugent chaque pixel isolément. Ils ne savent pas qu'un pixel entouré de forêt est probablement de la forêt. Cette information de voisinage est pourtant décisive, et c'est ce qu'apporte un réseau convolutif.

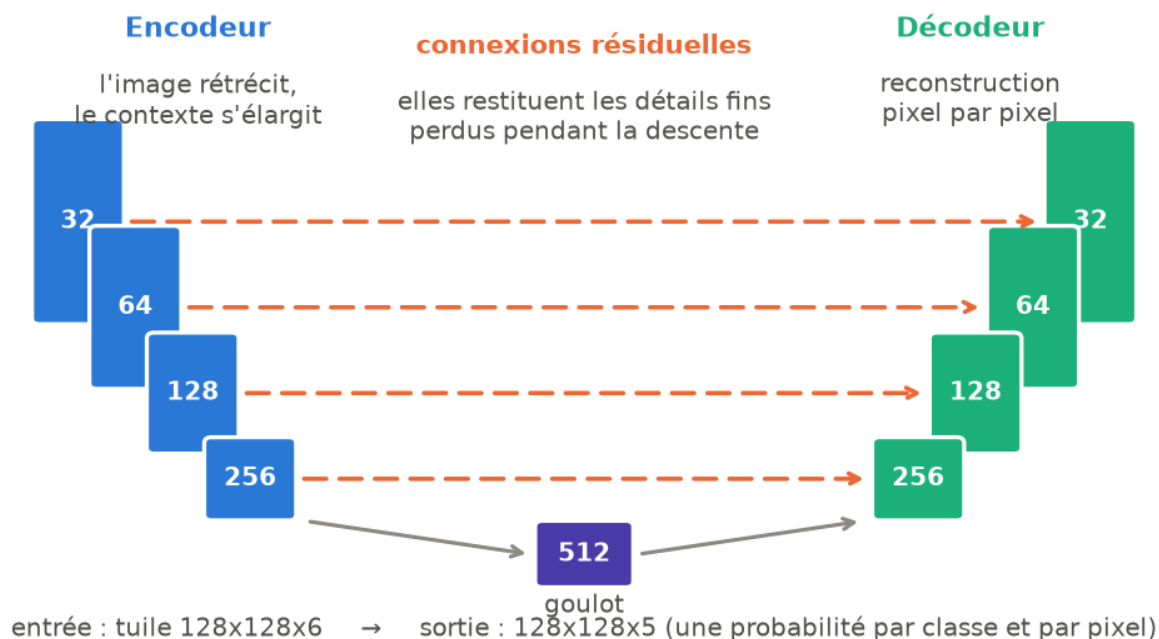


Figure 7. L'architecture U-Net : descente, goulot, remontée, et connexions résiduelles.

La descente, ou encodeur

L'image traverse des blocs de convolution qui extraient des motifs, entrecoupés de réductions de taille. À chaque niveau, l'image rétrécit mais le nombre de canaux augmente : on perd en précision spatiale et on gagne en compréhension du contexte. Les premiers niveaux détectent des textures, les derniers reconnaissent des formes entières, comme une parcelle agricole.

Le goulot d'étranglement

Au point le plus bas, l'image est réduite mais décrite par 512 canaux. C'est la représentation la plus abstraite, celle qui encode le sens de la scène plutôt que ses détails.

La remontée, ou décodeur

On reconstruit ensuite progressivement la résolution d'origine, jusqu'à produire une classification pixel par pixel. Sans précaution, cette reconstruction serait floue : les détails fins ont été perdus à la descente.

Les connexions résiduelles

C'est l'idée centrale du U-Net. À chaque niveau de la remontée, on reconcatène la carte correspondante de la descente, qui avait conservé le détail spatial. Le décodeur dispose ainsi à la fois du sens global, venu du goulot, et du détail local, venu directement de l'encodeur. C'est ce qui produit des contours de parcelles nets.

La sortie applique une fonction softmax sur cinq canaux : pour chaque pixel, le réseau fournit une probabilité par classe, dont la somme vaut 1. La classe retenue est celle de probabilité maximale, et les probabilités elles-mêmes indiquent le degré de confiance.

Si TensorFlow n'est pas installé, le code bascule sur une segmentation par centroïdes spectraux qui expose la même interface. Le pipeline reste exécutable de bout en bout, avec des performances moindres. C'est un repli assumé, tracé dans les journaux.

5.6 Le prédicteur de risque

Ce modèle répond à la question qui intéresse réellement un décideur : où faut-il intervenir avant que la forêt ne disparaisse.

Construction de l'étiquette

Elle est temporelle. On compare la carte de couverture de l'année N à celle de l'année N+1. Un pixel est étiqueté positif s'il était forêt en N et ne l'est plus en N+1. L'entraînement se restreint aux pixels encore forestiers ; sans cette restriction, le modèle apprendrait surtout à reconnaître ce qui est déjà détruit, ce qui n'a aucune valeur prédictive.

Les six variables explicatives

Variable	Justification
Distance à la route	Le défrichement suit les axes de pénétration
Distance au village	La pression vient des zones habitées
Distance à la zone déjà défrichée	Le front avance par contiguïté
Pente	Une pente forte décourage la mise en culture
Altitude	Corrèle avec l'accessibilité et le type de sol
Taux de déforestation du voisinage	Mesure la pression locale immédiate

Aucune de ces variables n'est spectrale. Le modèle ne regarde pas la couleur du pixel mais sa situation géographique. La logique sous-jacente est celle du front de déforestation : on ne défriche pas au hasard, on défriche à partir de ce qui est déjà ouvert et accessible.

Le risque se concentre sur la lisière : 7 049 pixels au-delà de 70 sur 100

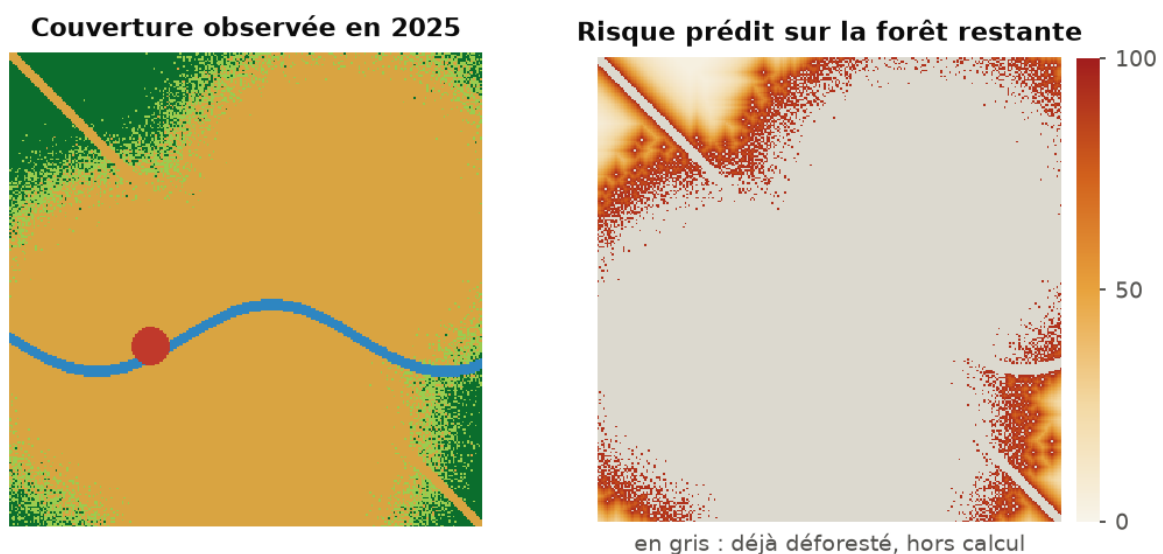


Figure 8. La couverture observée et le risque prédit sur la forêt restante.

Le résultat est lisible immédiatement : le risque se concentre sur les lisières et le long de la route diagonale. Le cœur des massifs encore intacts reste à faible risque. C'est ce que l'on attend d'un modèle de front, et

c'est un bon test de crédibilité.

Ce que l'application sert réellement

La carte de risque affichée par l'API, le dashboard et le frontend passe par `src/data/provider.py`, qui applique une règle explicite : si un modèle entraîné est présent dans `data/models/`, c'est lui qui prédit ; sinon, le système replie sur une référence géométrique où le risque décroît exponentiellement avec la distance au front de déforestation.

Les deux renvoient une grille 0 à 100 de même forme, donc l'appelant n'a rien à changer. La fonction `risk_source()` indique laquelle est active, et les interfaces affichent ce libellé sous la carte. Un jury qui demande d'où vient la carte obtient donc la réponse à l'écran, sans avoir à ouvrir le code.

La référence géométrique n'est pas un pis-aller : elle sert de point de comparaison. Un modèle appris qui ne ferait pas mieux qu'une simple décroissance avec la distance n'apporterait rien, et c'est exactement le genre de vérification qu'un travail sérieux doit exposer.

5.7 Le découpage entraînement / test, point méthodologique central

C'est le point sur lequel un jury interrogera, et celui où beaucoup de projets de télédétection se trompent.

Deux pixels voisins sur une image satellite se ressemblent énormément : même type de sol, même éclairage, souvent la même classe. Si l'on répartit les pixels aléatoirement entre entraînement et test, presque chaque pixel de test a un voisin immédiat dans l'ensemble d'entraînement. Le modèle retrouve alors au test des choses qu'il a déjà vues. Le score obtenu est excellent et ne veut rien dire.

On appelle cela une fuite de données géographique. La parade appliquée par le projet consiste à découper la grille en 64 blocs et à affecter chaque bloc entier, et non chaque pixel, à l'entraînement, à la validation ou au test.

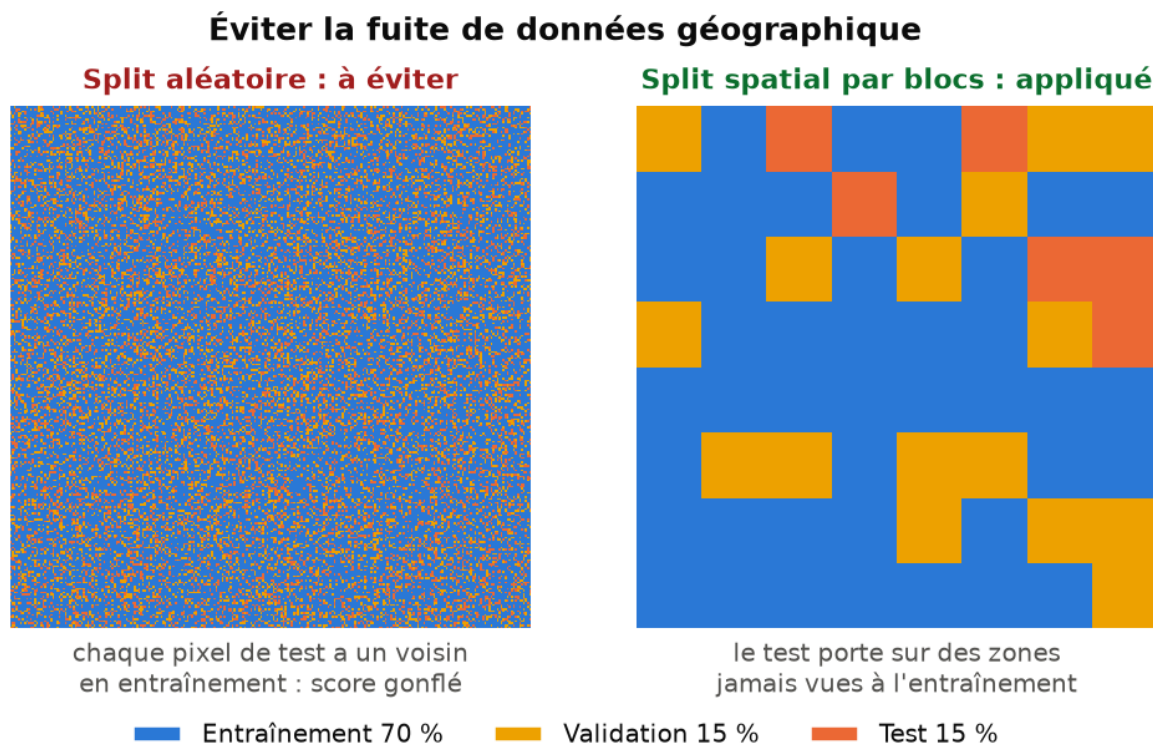


Figure 9. À gauche le découpage aléatoire, à éviter. À droite le découpage spatial par blocs, appliqué par le projet.

Avec le découpage par blocs, le modèle est évalué sur des zones géographiques entières qu'il n'a jamais rencontrées. Le score est plus faible qu'avec un découpage aléatoire, et c'est précisément ce qui le rend crédible : il mesure une vraie capacité de généralisation.

Pour le U-Net, le même raisonnement s'applique au niveau des tuiles : ce sont des tuiles entières qui partent en test, jamais des morceaux de tuile.

5.8 L'évaluation des modèles

Métrique	Définition courte	Pourquoi elle est suivie
Accuracy	Part de pixels correctement classés	Lisible, mais trompeuse ici : la forêt domine largement
Précision	Parmi les pixels prédits classe X, part de justes	Mesure les fausses alertes
Rappel	Parmi les pixels réellement X, part de retrouvés	Mesure les oublis
F1-macro	Moyenne harmonique, moyennée sans pondération	Une classe rare mal prédite fait chuter le score
Mean IoU	Intersection sur union, moyennée	Métrique de référence en segmentation d'images
AUC-ROC	Qualité du classement des probabilités	Indépendante du seuil de décision retenu

Le F1-macro et le Mean IoU sont les deux chiffres à mettre en avant. Un modèle qui prédirait bêtement forêt partout obtiendrait déjà une accuracy honorable, un F1-macro très mauvais, et serait immédiatement démasqué par la matrice de confusion.

Le rapport complet est écrit dans `data/processed/model_metrics.json` et servi par l'endpoint `/api/v1/models`.

5.9 Ce que le pipeline produit au final

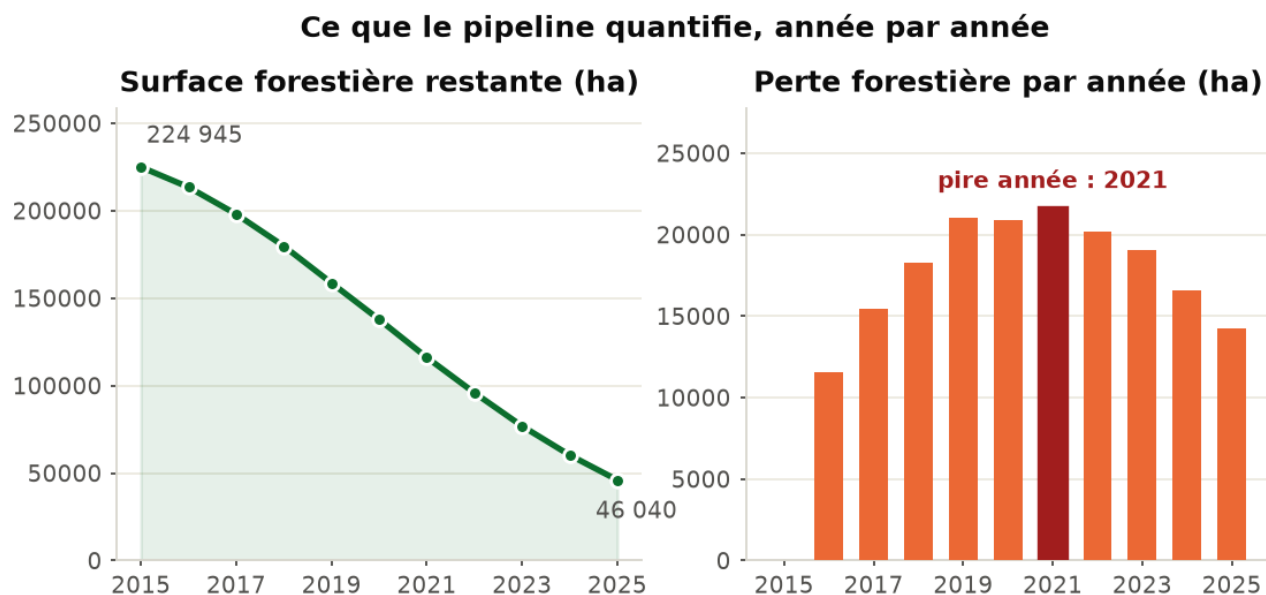


Figure 10. Les deux sorties quantitatives : le stock de forêt restant et le flux annuel de perte.

Le graphique de gauche donne le stock, celui de droite le flux. Les deux sont nécessaires : le stock dit combien il reste, le flux dit à quelle vitesse on le perd et permet d'identifier les années de rupture.

Ces chiffres sont calculés en comptant les pixels de classe 0 et 1 et en les multipliant par la surface d'un pixel. C'est une simple conversion d'unité, mais elle transforme une carte en argument chiffré, seul langage audible par une administration ou un bailleur.

Recul du couvert forestier sur la zone d'étude

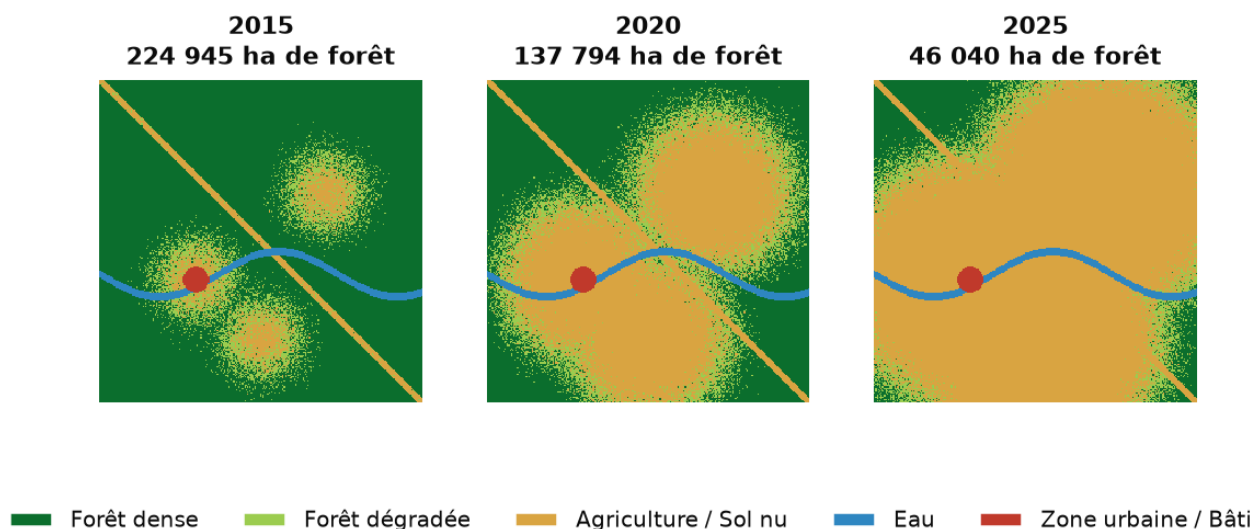


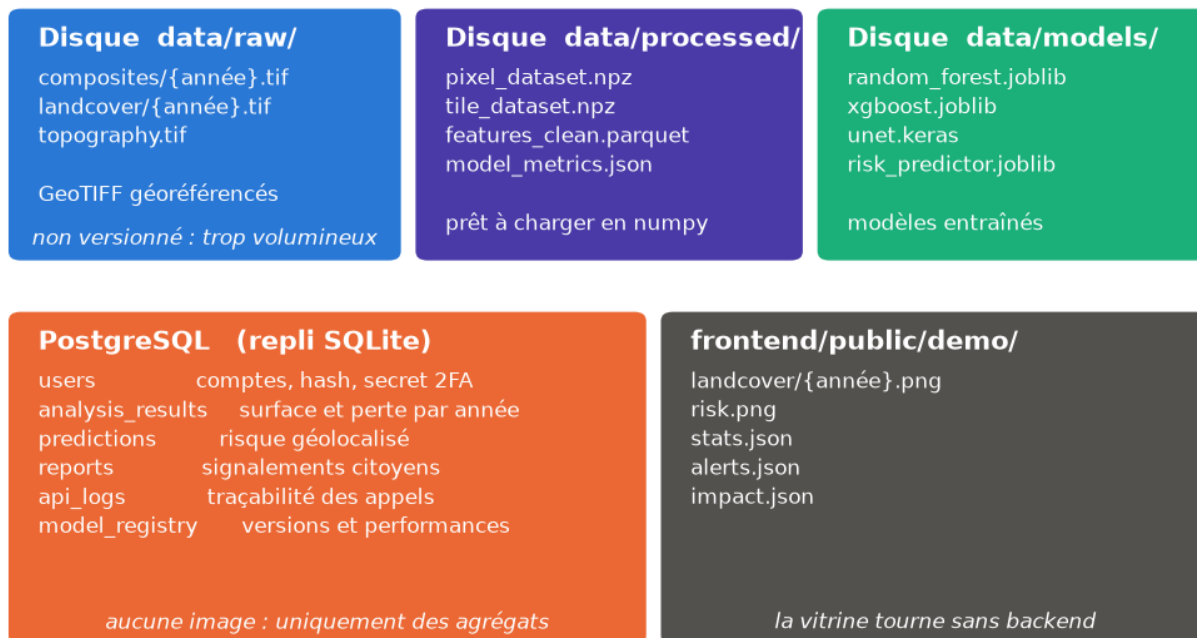
Figure 11. Le recul du couvert forestier sur la période étudiée.

Sur ces trois cartes, on lit la dynamique décrite plus haut : l'agriculture progresse en tache d'huile depuis les villages et le long de la route diagonale, en laissant une frange de forêt dégradée à l'avant du front. C'est exactement le motif que le prédicteur de risque exploite.

6. Le stockage des données

Le stockage est volontairement éclaté. Chaque type de donnée va là où son format est le plus efficace, plutôt que de tout entasser au même endroit.

Chaque donnée va là où son format est le plus efficace



Les pixels vivent sur le disque. La base ne garde que ce qui doit être interrogé, filtré ou joint.

Figure 12. Répartition des données entre le disque, la base relationnelle et les assets du frontend.

6.1 Les images : GeoTIFF sur le disque

Un GeoTIFF est un fichier TIFF qui embarque son géoréférencement : système de coordonnées, emprise, résolution. Chaque pixel du fichier correspond donc à une position réelle sur Terre, ce qu'un PNG ou un JPEG ne permettent pas.

```
data/raw/
  composites/
    2015.tif ... 2025.tif    6 bandes, reflectance Sentinel-2
  landcover/
    2015.tif ... 2025.tif    1 bande, classes 0 a 4 (verite terrain)
  topography.tif            3 bandes : altitude, pente, aspect
```

Contrainte forte : toutes les images doivent partager exactement la même grille, c'est-à-dire les mêmes dimensions, la même emprise et la même résolution. Sans cela, le pixel numéro 1 000 d'une image ne désignerait pas le même endroit que le pixel numéro 1 000 d'une autre, et toute comparaison temporelle serait faussée.

L'alignement est vérifié au chargement du dataset, et le script `scripts/check_real_data.py` contrôle l'ensemble du jeu de données avant l'entraînement : années détectées, nombre de bandes, cohérence des grilles, présence d'étiquettes.

6.2 Les datasets préparés

Une fois les features calculées et le découpage effectué, les tableaux sont écrits en NPZ compressé, le format natif de numpy. On y trouve `pixel_dataset.npz` pour les modèles par pixel et `tile_dataset.npz` pour le U-Net, accompagnés d'un `dataset_metadata.json` qui documente le nom des features et la taille de chaque partition.

L'intérêt est la vitesse : recharger un NPZ prend une fraction de seconde, alors que reconstruire les features depuis les GeoTIFF prend plusieurs minutes. On paie le calcul une fois.

La sortie du pipeline Spark est stockée à part, en Parquet partitionné par année. Le Parquet est un format colonne : lire uniquement la colonne NDVI de l'année 2023 ne demande pas de parcourir le reste du fichier.

6.3 Les modèles entraînés

Fichier	Modèle	Format
random_forest.joblib	Random Forest	Sérialisation joblib
xgboost.joblib	XGBoost	Sérialisation joblib
unet.keras	U-Net	Format natif Keras
unet_fallback.joblib	Repli centroïdes	Sérialisation joblib
risk_predictor.joblib	Prédicteur de risque	Sérialisation joblib

Sauvegarder un modèle entraîné évite de tout recalculer à chaque prédiction. L'API charge le fichier au démarrage et répond ensuite en quelques millisecondes.

6.4 La base de données relationnelle

PostgreSQL, accédé via SQLAlchemy (`src/api/database.py`). Elle ne stocke aucune image : uniquement des résultats agrégés, des métadonnées et les données applicatives.

Table	Colonnes principales	Rôle
users	email, password_hash, role, otp_secret	Comptes et authentification à deux facteurs
analysis_results	year, total_forest_ha, forest_loss_ha, rate	Résultats d'analyse par année
predictions	zone_lat, zone_lon, risk_score, model_version	Prédictions de risque géolocalisées
reports	lat, lon, description, severity, status	Signalements soumis par les citoyens
api_logs	user_id, endpoint, method, status_code	Traçabilité des appels à l'API
model_registry	model_name, version, accuracy, f1_score	Versions de modèles et performances associées

Le mot de passe n'est jamais stocké en clair, seulement son empreinte cryptographique, ce qui rend la valeur inutilisable même en cas de fuite de la base. La table `reports` mérite d'être signalée : elle relie l'observation satellite au terrain, en permettant à un habitant de signaler une coupe que le satellite n'a pas encore vue.

Si PostgreSQL est injoignable, la couche bascule automatiquement sur un SQLite local, dans `data/deforestwatch.db`. L'API démarre donc partout, y compris sur un poste sans serveur de base installé.

6.5 Pourquoi les images ne vont pas en base

- Volume : une seule année d'images pèse de plusieurs centaines de mégaoctets à plusieurs gigaoctets.
- Mode d'accès : le traitement lit des blocs entiers de pixels, jamais des lignes filtrées par condition.
- Outillage : rasterio, numpy et GDAL lisent le GeoTIFF nativement, avec accès partiel à une fenêtre géographique.
- Coût : une base relationnelle facture cher le stockage de données binaires volumineuses, pour un bénéfice nul ici.

La règle appliquée tient en une phrase : les pixels vivent sur le disque, la base ne garde que ce qui doit être interrogé, filtré ou joint.

6.6 Ce qui est versionné, et ce qui ne l'est pas

Dans Git	Hors Git
Code source, configuration, tests	data/raw/ : les images brutes
Documentation, notebooks, scripts	data/processed/ : les datasets dérivés
Assets de démonstration du frontend	data/models/ : les modèles entraînés
README et fichiers .gitkeep des dossiers de données	Le fichier .env et la clé de service GEE

La logique est double. D'un côté, Git n'est pas fait pour des fichiers binaires volumineux qui changent souvent. De l'autre, les secrets ne doivent jamais entrer dans l'historique d'un dépôt, car ils y resteraient consultables même après suppression.

Le dépôt reste donc léger et clonable rapidement, et tout ce qui est reconstituable se reconstruit par une commande.

7. La bascule démonstration / données réelles

C'est la pièce d'architecture qui tient l'ensemble, et celle qui mérite le plus d'être défendue.

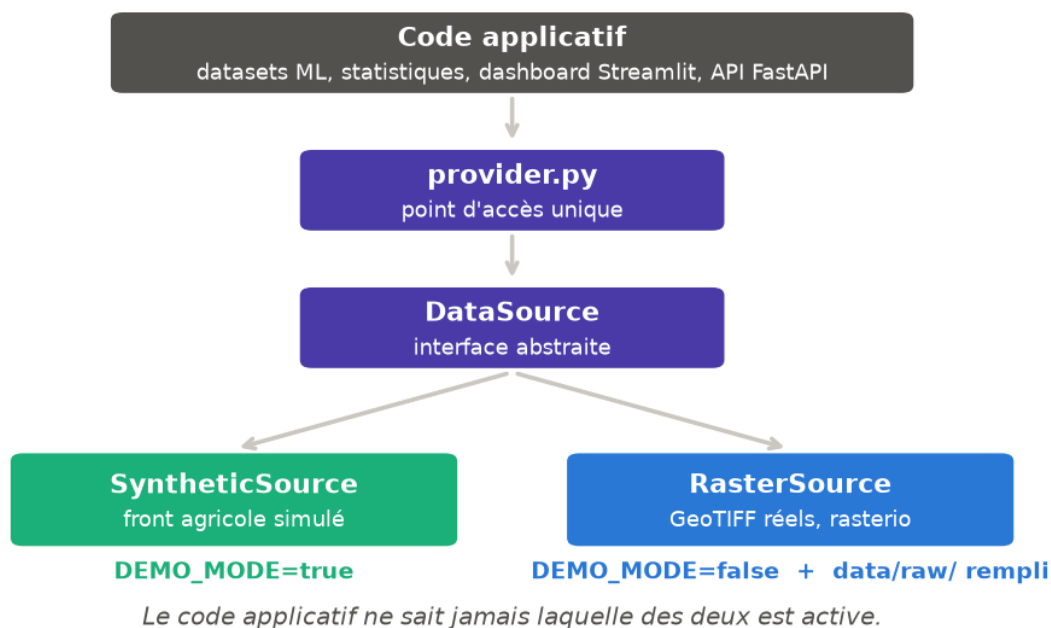


Figure 13. L'abstraction qui rend la source de données interchangeable.

`src/data/sources.py` définit une interface abstraite, `DataSource`, qui déclare quatre opérations : lister les années disponibles, fournir le composite d'une année, fournir la carte de classes, fournir la topographie. Deux classes l'implémentent.

Implémentation	Ce qu'elle fait
SyntheticSource	Simule un front agricole progressant depuis les villages et les routes, avec des signatures spectrales réalistes et du bruit.
RasterSource	Lit les vrais GeoTIFF de <code>data/raw/</code> avec <code>rasterio</code> et vérifie l'alignement des grilles.

Tout le code applicatif passe par `src/data/provider.py` et n'appelle jamais directement l'une ou l'autre. Il ignore laquelle est active.

La résolution automatique

À l'exécution, `resolve_source()` applique une règle simple : si `DEMO_MODE` vaut `false` et que des fichiers `.tif` sont présents dans `data/raw/composites/`, la source réelle est retenue. Sinon, on retombe sur la source synthétique, avec un avertissement explicite dans les journaux.

Un administrateur peut même basculer à chaud, sans redémarrer le service, via l'endpoint `POST /api/v1/admin/source/{mode}`, où `mode` vaut `auto`, `demo` ou `real`.

Conséquence pratique : déposer les GeoTIFF au bon format et passer `DEMO_MODE` à `false` suffit à faire passer l'ensemble du projet sur des données réelles, sans modifier une seule ligne de code. Datasets, statistiques, cartes, API et dashboard suivent automatiquement.

Une exception subsiste, à connaître par honnêteté : le pipeline Spark appelle encore directement le générateur synthétique et restera donc en mode démonstration même avec de vraies images en place. C'est le prochain point à corriger pour que la bascule soit complète.

8. Questions de soutenance

Les questions ci-dessous reviennent presque systématiquement sur ce type de projet. Les réponses sont là pour être reformulées avec vos mots.

Pourquoi la médiane et pas la moyenne pour le composite ?

Parce que la moyenne est sensible aux valeurs extrêmes. Un seul nuage très réfléchissant tirerait la moyenne vers le haut et blanchirait le pixel. La médiane ignore les extrêmes par construction : tant que plus de la moitié des dates sont dégagées, elle donne la valeur du sol.

Pourquoi trois modèles plutôt qu'un seul ?

Pour comparer deux familles d'approches sur le même problème. Random Forest et XGBoost travaillent pixel par pixel, sont rapides et interprétables. Le U-Net exploite le contexte spatial et produit des contours plus nets, au prix d'un entraînement plus lourd. La comparaison chiffrée fait partie du résultat, elle n'est pas un détour.

Comment savez-vous que votre modèle n'a pas simplement mémorisé ?

Grâce au découpage spatial par blocs. Le test porte sur des zones géographiques entières que le modèle n'a jamais vues. Avec un découpage aléatoire classique, chaque pixel de test aurait eu un voisin en entraînement et le score aurait été artificiellement élevé.

Que se passe-t-il si une année est entièrement nuageuse ?

Le composite optique de cette année est inexploitable. Deux solutions existent dans le projet : élargir la fenêtre temporelle au-delà de la saison sèche, ce qui dégrade la comparabilité, ou s'appuyer sur le radar Sentinel-1, qui traverse les nuages et reste disponible.

Pourquoi ne pas stocker les images dans PostgreSQL ?

Volume, mode d'accès et coût. Le traitement lit des blocs entiers de pixels, ce qu'une base relationnelle fait mal, tandis que les bibliothèques de télédétection lisent le GeoTIFF nativement, y compris partiellement. La base garde ce qu'on interroge et joint, c'est-à-dire des agrégats et des données applicatives.

Vos données sont synthétiques : quelle est la valeur du travail ?

La chaîne de traitement est réelle et complète : requêtes GEE, masquage SCL, composite médian, indices, découpage spatial, entraînement, évaluation, API, interfaces. Le mode démonstration ne remplace que la source d'images, derrière une interface abstraite. Déposer de vraies images bascule l'ensemble sans modification du code, ce qui est vérifiable en direct avec la commande `make export-demo`.

Quelle est la précision attendue en conditions réelles ?

Sur des composites Sentinel-2 réels et une vérité terrain Hansen, la littérature situe ce type de classification autour de 0,85 à 0,92 de F1-macro sur cinq classes. Les confusions se concentrent entre forêt dégradée et agriculture, parce que ces deux classes se recouvrent physiquement : une jachère en repousse est difficile à distinguer d'une forêt dégradée, même pour un oeil humain.

9. Aide-mémoire des commandes

Commande	Effet
make install	Installe les dépendances Python
make seed	Génère les données de démonstration
make export-demo	Écrit des GeoTIFF de test dans data/raw/
make check-data	Vérifie le jeu de données réelles
make real	Bascule la configuration en mode réel
make demo	Revient au mode démonstration
make mode	Affiche le mode de données courant
make train	Entraîne les modèles et produit les métriques
make test	Lance la suite de tests avec couverture
make api	Démarre l'API FastAPI sur le port 8000
make dashboard	Démarre le dashboard Streamlit sur le port 8501
make frontend	Démarre le frontend React sur le port 5173
make synthese	Régénère ce document et ses figures

Les fichiers à connaître

Fichier	Ce qu'il contient
config/settings.py	Toute la configuration : zone, classes, bandes, hyperparamètres
src/data/gee_collector.py	Les requêtes Google Earth Engine
src/data/sources.py	L'abstraction démonstration / réel
src/data/dataset_builder.py	Le découpage spatial et la construction des datasets
src/models/trainer.py	Le pipeline d'entraînement unifié
src/models/evaluator.py	Le calcul des métriques et la comparaison
src/api/database.py	Le schéma de la base de données
scripts/gee_export.py	L'export des images vers data/raw/

Document généré par scripts/generate_synthese_technique.py. Les figures sont produites par scripts/figures_synthese.py à partir du code du projet. Pour les régénérer après une modification : make synthese.